

[< Go to the original](#)

How I Built a Multi-Agent AI Workflow That Automates 80% of My Data Pipeline

Every Monday morning used to look the same.



Yash Batra

Follow

androidstudio · February 17, 2026 (Updated: February 17, 2026) · Free: No

A new CSV would land in S3. I'd download it, scan for missing columns, compare it against our canonical schema, write a quick transformation script, sanity-check aggregates, and then draft a short summary for stakeholders. None of it was intellectually difficult. It was just repetitive. By Thursday, I'd do it again for a different source.

and over.

Eventually, I stopped optimizing SQL queries and started optimizing the decision loop.



This is a reproducible walkthrough of the multi-agent AI workflow I built to automate about 80% of my data pipeline glue work. Not full autonomy. Not hype. Just tightly scoped automation around boring, repeatable tasks.

The Problem: Human Glue Work

Before automation, here's what my weekly pipeline looked like:

- Validate new data drops (schema drift, required columns)
- Map new fields to canonical schema
- Suggest simple transformations

• Write a short summary report

Time spent per week: ~10 hours.

Not because the system was fragile. Because humans were acting as glue.

The drain wasn't technical difficulty. It was:

- Re-evaluating schema mismatches
- Deciding whether `total_amt` maps to `total_amount`
- Re-checking if a 17% spike is legitimate or broken
- Writing similar reports repeatedly

That's the kind of structured, repeatable decision-making where agentic systems actually make sense.

The Architecture: Small Agents, Clear Boundaries

I didn't build a "super-agent." I built small agents with narrow responsibilities.

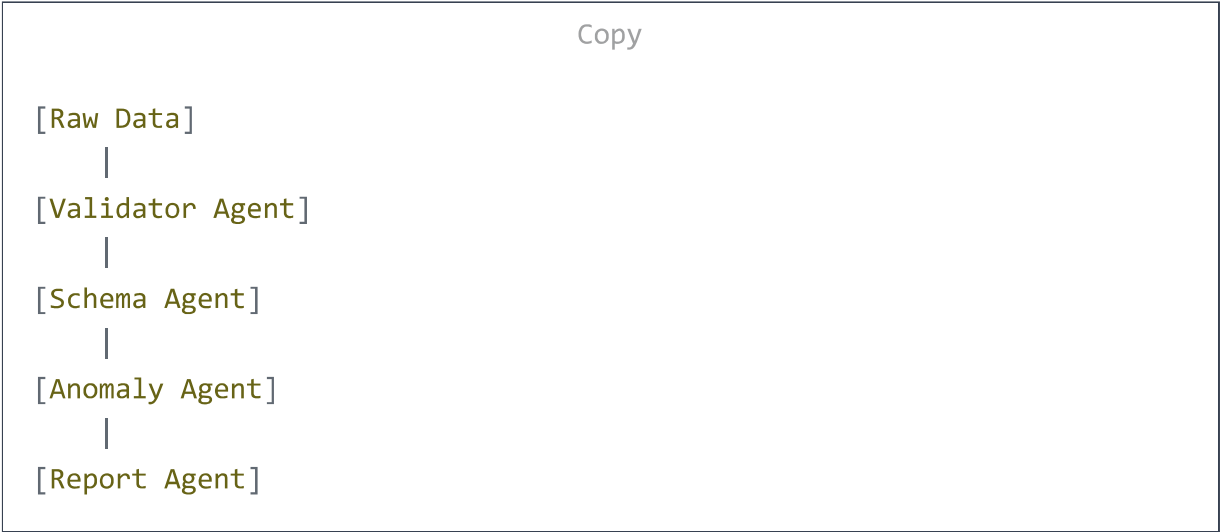
1. Ingestion Validator Agent
2. Schema Mapping Agent
3. Anomaly Detection Agent
4. Reporting Agent

No agent owns the workflow.

Each one:

- Accepts structured input

ASCII Diagram: Multi-Agent Flow



Each stage is deterministic. The orchestration layer decides order and retries.

This isn't autonomous chaos. It's structured flow.

The Orchestrator: Control Without Chaos

The orchestrator is intentionally boring.

It handles:

- Ordering
- Retries
- Schema validation
- Escalation

Here's a simplified version in Python:

```
def run_pipeline(data):  
    validated = validator.run(data)  
    mapped = schema_agent.run(validated)  
    anomalies = anomaly_agent.run(mapped)  
    report = reporter.run(anomalies)  
  
    return report
```

In production, each `.run()` :

- Validates JSON schema
- Applies retry policy
- Logs prompts + responses
- Tracks latency

The orchestrator never delegates control. Agents don't call each other. All handoffs are explicit.

Agent 1: Ingestion Validator

Responsibility:

- Check required columns
- Detect schema drift
- Flag malformed records

It receives:

Copy

```
{  
  "columns": ["order_id", "total_amt", "created_at"],  
  "sample_rows": [...]  
}
```

Copy

```
prompt = f"""
You are a data ingestion validator.

Required columns: {required_columns}
Incoming columns: {incoming_columns}

Return JSON:
{{
  "missing_columns": [],
  "unexpected_columns": [],
  "is_valid": true|false
}}
```

Parsing with strict schema:

Copy

```
from pydantic import BaseModel

class ValidationResult(BaseModel):
    missing_columns: list[str]
    unexpected_columns: list[str]
    is_valid: bool

response = llm.invoke(prompt)
result = ValidationResult.model_validate_json(response)
```

If `is_valid` is false, the pipeline stops.

No guessing. No auto-fixes.

Agent 2: Schema Mapping Agent

Responsibility:

Example output:

Copy

```
{
  "mappings": {
    "total_amt": "total_amount"
  },
  "transformations": {
    "total_amount": "float(value)"
  },
  "confidence": 0.92
}
```

Guardrails:

- Canonical schema is fixed and injected
- Agent cannot invent new canonical fields
- Confidence < 0.8 triggers human review

Example invocation:

Copy

```
mapping = schema_agent.run({
    "incoming_columns": incoming_columns,
    "canonical_schema": canonical_schema
})

if mapping.confidence < 0.8:
    escalate_to_human(mapping)
```

This is where hallucinations happened early on. The guardrails fixed that.

Agent 3: Anomaly Detection Agent

This agent has two layers:

1. Statistical pre-check (code)
2. Reasoning layer (LLM)

First, deterministic thresholds:

Copy

```
def precheck(metrics):  
    alerts = []  
    if metrics["daily_revenue_change"] > 0.25:  
        alerts.append("Revenue spike > 25%")  
    return alerts
```

Then contextual reasoning:

Copy

```
prompt = f"""  
Metrics:  
{metrics}  
  
Existing alerts:  
{alerts}  
  
Explain whether anomalies appear legitimate.  
Return JSON:  
{{  
    "summary": "",  
    "severity": "low|medium|high",  
    "confidence": 0-1  
}}  
"""
```

Fallback behavior:

Freedium

```
result = anomaly_agent.run(data)

if result.confidence < 0.7:
    result.severity = "needs_review"
```

The agent never decides alone. It layers reasoning on top of deterministic checks.

Failure Handling and Guardrails

Every boundary validates output.

ASCII Diagram: Guardrail Loop

Copy

```
[Agent Output]
  |
  Validate
  |
  Retry / Escalate
```

Policies:

- 2 automatic retries on malformed JSON
- Strict Pydantic validation
- Escalation on low confidence
- Versioned prompts
- Logged inputs + outputs

This turned brittle prompts into controlled components.

Before vs After: The Real Impact

- Manual time: 10 hrs/week → 2 hrs/week
- Schema mapping errors: ~12% → ~3%
- Review cycles: reduced from 3 rounds to 1
- Throughput: ~2× more data sources onboarded monthly

The biggest gain wasn't speed. It was consistency.

The system makes the same decision the same way every time.

What Broke

This wasn't smooth.

Early failures:

- Hallucinated canonical fields
- Overconfident mappings
- Prompt brittleness when column names were weird
- JSON formatting drift

Fixes:

- Explicit schema injection
- Confidence scoring
- Strict JSON parsing
- Deterministic pre-check layers
- Hard boundaries between agents

Why This Works for Pipelines

Pipelines are structured.

They have:

- Known schemas
- Repeated tasks
- Validatable outputs
- Deterministic stages

That makes them ideal for a multi-agent AI workflow.

Agentic systems fail when tasks are ambiguous and unbounded.

They succeed when autonomy is constrained and outputs are verifiable.

This is reproducible AI workflow code applied to boring decisions.

Where This Would Fail

This approach breaks down in:

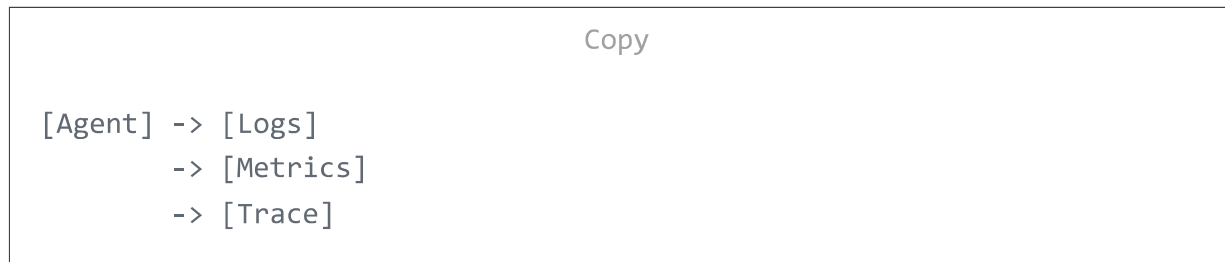
- High-ambiguity domains (legal interpretation)
- Compliance-heavy workflows with liability
- Ultra-low-latency systems
- Real-time transactional processing

It's not for creative reasoning or open-ended judgment.

Observability for Agents

Agents without observability become impossible to debug.

ASCII Diagram: Observability Layer



What we track:

- Prompt version
- Input payload hash
- Output JSON
- Latency per agent
- Retry count
- Escalation rate

We also classify errors:

- Parse failures
- Low-confidence flags
- Drift detection mismatches

This made the system maintainable.

The Design Principle That Made It Work

1. Small agents

2. Deterministic orchestration

3. Validation at every boundary

No agent owns the pipeline.

No implicit chaining.

No silent failure.

This isn't about LLM agents backend hype. It's about reducing repetitive decision-making in structured systems.

Final Reflection

The workflow doesn't eliminate engineers.

It eliminates the part where an engineer has to decide, for the fiftieth time, whether `total_amt` probably means `total_amount`.

The most powerful automation here isn't intelligence.

It's reducing the number of times a human has to make the same decision twice.